

限的分发会通过一个用户态的服务——命名服务（Naming Service）——来处理。命名服务进程则指代提供该服务的具体进程。

什么是命名服务呢？命名服务像是一个全局的看板，可以协调服务端进程和客户端进程之间的信息。简单来说，服务端进程可以将自己提供的服务告诉命名服务进程，比如文件系统进程可以注册一个“文件系统服务”，网络系统进程可以注册一个“网络服务”。而客户端进程可以在命名服务中查询当前服务，并选择自己希望建立连接的服务去尝试获取权限。具体是否分发权限给客户端进程，是由命名服务和对应的服务端进程根据特定的策略来判断的。比如，文件系统进程的策略可能是，允许命名服务将连接文件系统的权限任意分发，这样所有的进程都可以使用全局的文件系统。而数据库进程的策略是，客户端必须提供特定私钥签名的证书，并且证书符合数据库进程的要求，才能完成建立。使用命名服务有很多好处，比如各个服务不再是内核中的 ID 等抽象的表示，而是对应用更友好的“名字”。前面章节中提到过的“服务发现”，通常就是使用命名服务来实现的。

为什么命名服务一般在用户态？从上面的介绍中可以看到命名服务的功能其实并不简单，并且通常需要支持很多不同的策略。将这些策略实现在内核中，对于微内核系统显然是不合适的。而即使是宏内核系统，这也不是一种优雅的设计。举例来看，宏内核的 Android Binder、ROS 以及微内核的 L4 等，都是依赖（用户态的）命名服务来完成连接权限分发的。当然，在具体的设计细节上，它们仍然存在很多的不同。

除了命名服务外，另外一种常见的方式是通过继承来分发连接权限。比如在 Linux 中，匿名管道通常用于父子进程之间的通信，内核通过在 fork 操作的时候复制文件描述符表来建立父子进程间的连接。微内核系统在创建新的线程 / 进程时，内核通常也允许父进程将特定的 Capability 直接继承给子进程。这种继承的方式对于一些不希望暴露到全局的通信连接来说是一个保密性更好的方式，不过它的适用场景相对也比较有限。

8.1.11 总结

图 8.5 给出了宏内核操作系统中常见的进程间通信机制的对比。可以看到，虽然在 IPC 的几个设计角度上这些方案各有异同，但是它们之间的主要区别是在数据抽象上。例如，管道提供字节流的数据抽象，并通过文件抽象（即文件描述符）暴露接口给应用使用，而共享内存则使用内存区间作为数据抽象，并通过内存抽象（访存操作）供应用使用。在实际的应用中，虽然多种 IPC 方案都可以作为通信的选择，但是应用程序往往会根据对于数据抽象的需求来选择具体的方案。后续章节将围绕管道、共享内存和消息队列介绍 IPC 机制。

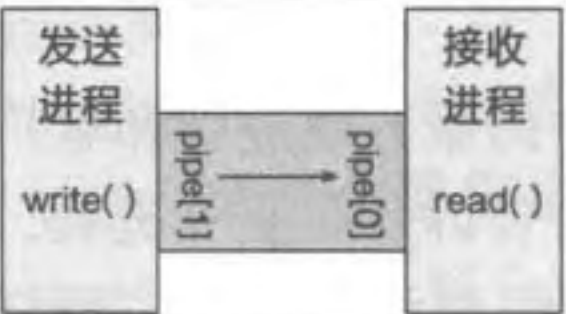
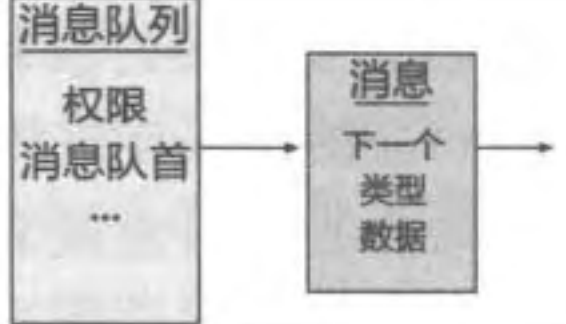
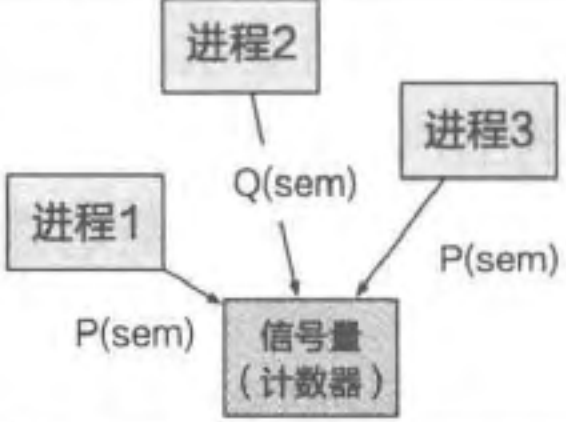
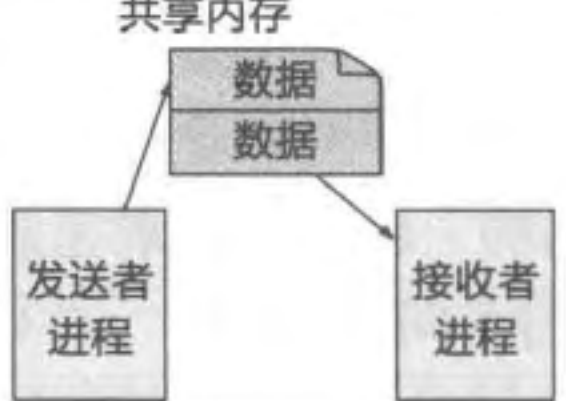
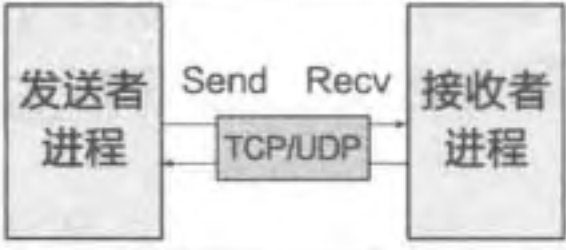
	IPC 机制	数据抽象	参与者	方向	内核实现
	管道	字节流	两个进程	单向	通常以 FIFO 的缓冲区来管理数据。有匿名管道和命名管道两类主要实现
	消息队列	消息	多进程	单向 / 双向	队列的组织方式。通过文件的权限来管理对队列的访问
	信号量	计数器	多进程	单向 / 双向	内核维护共享计数器。通过文件的权限来管理对计数器的访问
	共享内存	内存区间	多进程	单向 / 双向	内核维护共享的内存区间。通过文件的权限来管理对共享内存的访问
	信号	事件编号	多进程	单向	为线程 / 进程维护信号等待队列。通过用户 / 组等权限来管理信号的操作
	套接字	数据报文	两个进程	单向 / 双向	基于 IP / 端口和基于文件路径两种寻址方式。利用网络栈来管理通信

图 8.5 宏内核进程间通信机制的对比

8.2 文件接口 IPC：管道

本节主要知识点

- ❑ 管道进程间通信是如何设计的？
- ❑ Linux 内核中的管道是如何实现的？
- ❑ 命名管道和匿名管道的区别是什么？

管道 (Pipe) 是宏内核场景下重要的进程间通信机制。顾名思义, 管道是两个进程间的一条通道, 一端负责投递, 另一端负责接收。举一个简单的例子, 我们经常会通过 `ps aux | grep target` 来查看当前是否有关键字 `target` 相关的进程在运行。这里其实是两个命令, 通过 `shell` 的管道符号 “|”, 将第一个命令的输出投递到一个管道, 而管道对应的出口是第二个命令的输入。`shell` 的管道符号 “|” 通常是利用操作系统提供的管道进程间通信机制来实现的, 如在 `Linux` 系统中, 可以通过 `pipe` 系统调用让应用创建一个管道, 并获取管道两端的读和写的两个端口。`shell` 可以通过将两个命令的标准输出 (`stdout`) 和标准输入 (`stdin`) 的文件描述符分别配置为管道的两端来实现两个命令的串联, 如上述的例子中, `ps` 命令的标准输出会被配置为管道的写端口, 而 `grep` 命令的标准输入会被配置为管道的读端口。通过管道这种方式, `ps` 和 `grep` 这两个命令对应的进程进行了一次协同合作。

管道是单向的 IPC, 内核中通常有一定的缓冲区来缓冲消息, 而通信的数据是字节流, 需要应用自己对数据进行解析, 比如抽象或封装为消息。如图 8.6 所示, 一个管道有且只能有两端, 而这两端一定是一个负责输入 (发送数据)、一个负责输出 (接收数据) 的。

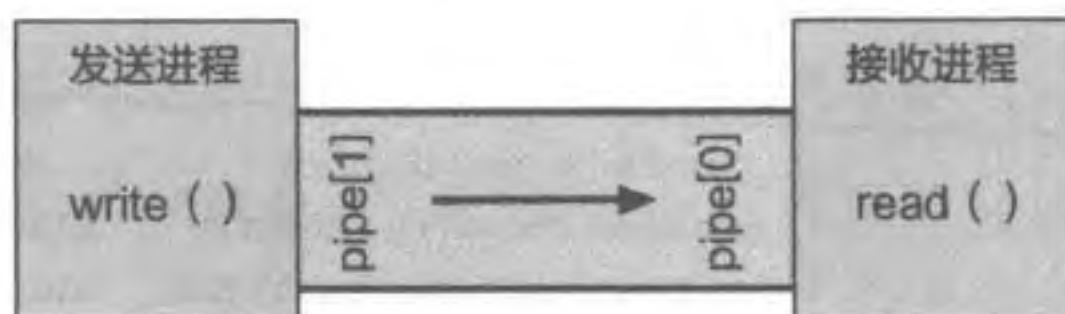


图 8.6 UNIX 下的管道

8.2.1 Linux 管道使用案例

代码片段 8.7 展示了一个 `Linux` 手册中给出的管道使用案例。该程序会通过进程 `fork` 创建一个子进程, 父进程会从命令行中读入一个字符串, 并通过管道将字符串传递给子进程, 而子进程将会从管道中读出该字符串并将内容打印出来。在这个案例中可以看到, 在管道通过 `pipe(pipefd)` 创建之后, 会得到两个文件描述符, 后续对管道的使用和对文件的使用几乎是完全相同的。案例中, 父进程通过 `write` 将数据内容写入管道, 而子进程通过 `read` 将数据从管道中读出。父、子进程都通过 `close` 来关闭管道端口。

代码片段 8.7 Linux 手册中的管道使用案例 (简化版)^[7]

```

1 #include <sys/types.h>
2 #include <sys/wait.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <string.h>
7
8 int main(int argc, char *argv[])
9 {
10     int pipefd[2];
11     pid_t cpid;
  
```

```
12  char buf;
13
14  if (argc != 2) {
15      fprintf(stderr, "Usage: %s <string>\n", argv[0]);
16      exit(EXIT_FAILURE);
17  }
18
19  if (pipe(pipefd) == -1) {
20      perror("pipe");
21      exit(EXIT_FAILURE);
22  }
23
24  cpid = fork();
25  if (cpid == -1) {
26      perror("fork");
27      exit(EXIT_FAILURE);
28  }
29
30  if (cpid == 0) {          // 子进程：从管道中读取数据
31      close(pipefd[1]);    // 关闭管道写端口
32      while (read(pipefd[0], &buf, 1) > 0)
33          write(STDOUT_FILENO, &buf, 1);
34      write(STDOUT_FILENO, "\n", 1);
35      close(pipefd[0]);
36      _exit(EXIT_SUCCESS);
37  } else {                 // 父进程从输入中获取一个字符串
38      close(pipefd[0]);    // 关闭管道读端口
39      write(pipefd[1], argv[1], strlen(argv[1]));
40      close(pipefd[1]);    // 管道读者会看到 EOF
41      wait(NULL);
42      exit(EXIT_SUCCESS);
43  }
44 }
```

该案例显示，在管道中我们可以通过文件接口来实现进程之间通信的目的。

在具体实现上，管道在 UNIX 系列的系统中会被当作一个文件。内核会为用户态提供代表管道的文件描述符，让其可以通过文件系统相关的系统调用来使用。管道的特殊之处在于，它的创建会返回一组（两个）文件描述符，在上述案例中 `pipe` 会同时返回两个文件描述符，放在 `pipefd` 中。不过实际上管道并不会使用存储设备，而是使用内存作为数据的缓冲区。这是因为管道本质上是为了实现通信的，一方面对于可持久化没有要求，另一方面还需要保证数据传输的高性能。

管道的行为和 FIFO (First-In-First-Out)^① 队列非常像，最早传入的数据会最先被读出来。一个进程输入数据后，另一个进程可以通过管道读到数据。如果还没有数据写入，

① 在一些 UNIX 系统中，会将后文中介绍的命名管道直接称为 FIFO。

拿到输出端的进程就开始尝试读数据，此时有两种情况：如果系统发现当前没有任何进程有这个管道的写端口，则会看到 EOF (End-Of-File)；否则阻塞在这个系统调用上，等待数据的到来。这里之所以存在第一种情况（没有任何进程有该管道的写端口），是因为管道的两个端口在 UNIX 系列的内核中是以两个独立的文件描述符的形式存在的，写端口有可能被进程主动关闭了。在第二种情况下，进程可以通过配置非阻塞选项来避免阻塞。

8.2.2 Linux 中管道进程间通信的实现

本节以 Linux 内核为例介绍管道的创建、读、写这三个操作的具体实现。

管道的创建是由 pipe 系统调用完成的，这个系统调用会返回两个文件描述符，对应管道的两端。系统调用的实现如代码片段 8.8 所示。

代码片段 8.8 Linux 5.10 中创建管道的系统调用核心代码（本节中无特殊说明的代码均基于 Linux 5.10）

```
1 // 合并了系统调用的声明和 do_pipe2 函数
2 SYSCALL_DEFINE2(pipe2, int __user *, fildes, int, flags)
3 {
4     struct file *files[2];
5     int fd[2];
6     int error;
7
8     error = __do_pipe_flags(fd, files, flags);
9     if (!error) {
10         if (unlikely(copy_to_user(fildes, fd, sizeof(fd)))) {
11             fput(files[0]);
12             fput(files[1]);
13             put_unused_fd(fd[0]);
14             put_unused_fd(fd[1]);
15             error = -EFAULT;
16         } else {
17             fd_install(fd[0], files[0]);
18             fd_install(fd[1], files[1]);
19         }
20     }
21     return error;
22 }
```

在上述实现中，Linux 内核通过 __do_pipe_flags 创建管道对应的缓冲区（在 Linux 中会通过特殊的文件系统来实现）。创建的两个文件描述符分别为 O_RDONLY 和 O_WRONLY，这意味着两个文件描述符只能一个为写端口、一个为读端口。文件描述符成功创建后，内核将文件描述符返回给用户态程序，即代码片段中的 copy_to_user 以及 fd_install 函数。整个创建过程返回后，用户态程序拿到两个文件描述符，这两个文件描述符即对应创建好的管道的两个端口。用户态程序可以通过文件接口来使用这两个文件描述符，从而实现进程间通信。

那么这个管道真实的存储空间在哪里呢？Linux 中通过 `pipe_inode_info` 这个结构体来管理管道在内核中的信息，如代码片段 8.9 所示。在这个结构体中，内核会维护 `bufs` 的管道缓冲区，由其来保存通信的数据。

代码片段 8.9 管道在 Linux 内核中的结构（简化版）

```

1 struct pipe_inode_info {
2     struct mutex mutex;                // 保护管道
3     wait_queue_head_t rd_wait, wr_wait; // 读者和写者的等待队列
4     unsigned int head;                 // 缓冲区头
5     unsigned int tail;                 // 缓冲区尾
6     unsigned int readers;               // 并发读者数
7     unsigned int writers;               // 并发写者数
8     struct pipe_buffer *bufs;           // 管道缓冲区
9 };

```

在 Linux 中，管道的读写操作和普通的文件读写操作一样。用户程序通过 `read` 和 `write` 系统调用来读写管道内的数据。在 Linux 系统的设计中，这两个系统调用会最终调用到管道实现中注册的文件操作 `pipe_read` 和 `pipe_write`。管道读和管道写的实现十分类似，我们以管道读为例来进行介绍，如代码片段 8.10 所示。

代码片段 8.10 Linux 中管道的读操作（简化）

```

1 static ssize_t
2 pipe_read(struct pipe_inode_info *pipe, struct user_buffer *to)
3 {
4     // 获取用户态的缓冲区大小
5     size_t total_len = buffer_count(to);
6     ssize_t ret;
7
8     // 如果缓冲区大小为 0，直接返回
9     if (unlikely(total_len == 0))
10         return 0;
11
12     ret = 0;
13     // 锁住管道，对应 pipe_inode_info 中的 mutex
14     pipe_lock(pipe);
15
16     for (;;) {
17         unsigned int head = pipe->head;
18         unsigned int tail = pipe->tail;
19         unsigned int mask = pipe->ring_size - 1;
20
21         if (!pipe_empty(head, tail)) {
22             struct pipe_buffer *buf = &pipe->bufs[tail & mask];
23             size_t chars = buf->len;
24
25             if (chars > total_len) {
26                 chars = total_len;

```

```
27     }
28
29     copy_page_to_user_buffer(buf->page, buf->offset, chars, to);
30     ret += chars;
31     buf->offset += chars;
32     buf->len -= chars;
33
34     if (!buf->len) {
35         release_pipe_buf(pipe, buf);
36         tail++;
37         pipe->tail = tail;
38     }
39     total_len -= chars;
40     if (!total_len)
41         break;
42 }
43 // 没有数据，阻塞等待（或直接返回错误信息）
44 ...
45 }
46 pipe_unlock(pipe); // 释放管道锁
47 // ...
48 }
```

为了从缓冲区中读取数据，首先内核会锁住整个管道，避免在读的过程中发生管道状态的变化。随后，内核尝试从缓冲区中读取数据。如果当前有数据，那么内核会在读取到足够的数据后返回。如果当前没有数据，那么内核会尝试唤醒等待的写者，让其开始写入数据，并且使自己陷入阻塞状态。在进入阻塞状态前需要释放管道锁，否则写者即使被唤醒也无法进行相应的操作。当写者完成写操作后，会唤醒当前等待的读者，使其开始尝试新一轮的读操作。管道写和管道读的过程十分类似。

8.2.3 命名管道和匿名管道

管道的连接是如何建立的呢？在经典的 UNIX 实现中，管道通常有两类——命名管道和匿名管道，主要区别在于它们的创建方式。匿名管道是通过 `pipe` 系统调用创建的，在创建的同时进程会拿到读写的端口（两个文件描述符）。由于整个管道没有全局的“名字”，因此只能通过这两个文件描述符来使用它。这种情况下，通常要结合 `fork` 来使用，即用继承的方式来建立父子进程间的连接。具体过程是，父进程首先通过 `pipe` 创建对应的管道的两端，然后通过 `fork` 创建子进程。由于子进程可以继承文件描述符，父子进程相当于通过 `fork` 的继承完成了一次 IPC 权限的分发。后续父子进程就可以通过管道来进行进程间的通信。要注意的是，在完成继承后，其实父子进程会同时拥有管道的两端，此时需要父子进程主动关闭多余的端口，否则可能导致通信出错。这种连接方式对于父子进程等有着创建关系的进程来说比较方便，但是对于两个关系较远的进程就不太适用。

命名管道可以解决这个问题。命名管道是由另一个命令 `mkfifo` 来创建的。创建过程中会指定一个全局的文件名，由这个文件名（如 `/tmp/namedpipe`）来指代一个具体的管道（即管道名）。通过这种方式，只要两个进程通过一个相同的管道名进行创建（并且都拥有对其的访问权限），就可以实现在任意两个进程间建立管道的通信连接。

练习

8.6 管道的使用中是否包含前面介绍的命名服务？请解释理由。

8.7 管道属于间接通信还是直接通信？请解释理由。

8.3 内存接口 IPC：共享内存

本节主要知识点

□ 基于共享内存的通信机制需要内核提供怎样的支持？

□ 基于共享内存的单生产者单消费者是如何实现的？

为什么需要共享内存？共享内存存在本章开头的例子中已经使用到了，本节将进一步介绍共享内存的内部结构设计和接口。使用共享内存的很重要的一个原因是性能。其他进程间通信机制，包括消息队列（下节介绍）、管道等，都依赖内核提供完整的缓冲数据、接收消息、发送消息等一系列进程间通信接口。虽然这些完善的抽象方便了用户进程的使用，但其中涉及的数据拷贝和控制流转移等处理逻辑影响了这些抽象的性能。共享内存的思路其实是内核为需要通信的进程建立共享区域。一旦共享区域建立完成，内核基本上不需要参与进程间通信。通信的多方间既可以直接使用共享区域上的数据，也可以将共享区域当成消息缓冲区。大部分微内核系统都支持共享内存机制。本节将介绍 System V 共享内存。

8.3.1 共享内存

共享内存的核心思路其实很简单：共享内存允许两个或多个进程在其所在虚拟地址空间中映射相同的物理内存页，从而进行通信。本节以 Linux 中的设计为例（如图 8.7 所示）来介绍一些细节的设计和实现。

首先，内核会为全局所有的共享内存维护一个全局的队列结构，也即图中的共享内存队列。这个队列的每一项（`shmid_kernel` 结构体）都是和一个 IPC key 绑定的，这和其他 System V 的 IPC 机制类似。进程可以通过同样的 key 来找到并使用同一段共享内存区域。虽然这样的 key 是全局唯一的，但是能否使用这段共享内存，是通过 System V